FormAssist — Test Website

# Mumbai University

## Scholarship Portal

*Developer Guide — Frontend & Backend*

---

*Purpose: Simulated scholarship form to test the FormAssist AI autofill system*

*This is a standalone Flask web application — separate from the main FormAssist project*

# 1. Purpose of This Website

This is a fake but realistic-looking scholarship website built specifically for testing the FormAssist project. It simulates the Mumbai University Dr. Babasaheb Ambedkar Merit-cum-Means Scholarship portal.

The FormAssist team will use this site to demonstrate that their AI can:

- Extract Aadhaar and PAN card numbers from uploaded document images using OCR
- Auto-fill those numbers into the correct fields on this form
- Validate field formats before submission
- Compress uploaded document files to meet size restrictions

> ⚠ *This site does not connect to any real university system. No real data is submitted anywhere. It is for demonstration and testing only.*

---

# 2. What the Site Must Look Like

The site should look convincingly like a real government/university scholarship portal. Use a professional, formal design with the following visual style:

- Colour scheme: Deep navy blue (#0D2137), white, and gold/amber accent (#C9A84C)
- Font: A formal serif or government-style sans-serif — suggest Google Font 'Noto Serif' for headings
- Header: Mumbai University logo placeholder, site name, and a top navigation bar
- Footer: Disclaimer text saying 'This is a test portal for academic purposes only'
- Multi-step form with a visible step progress bar at the top
- Each step should save progress so the user can go back without losing data

## Pages to Build

| Page / Route | File | Description |
|---|---|---|
| / (Home) | index.html | Landing page with scholarship details, eligibility criteria, and Apply Now button |
| /apply (Step 1) | apply_step1.html | Personal details form |
| /apply/step2 | apply_step2.html | Academic details form |
| /apply/step3 | apply_step3.html | Bank and financial details form |
| /apply/step4 | apply_step4.html | Document upload page (Aadhaar, PAN, etc.) |
| /apply/step5 | apply_step5.html | Review all entered data before final submission |
| /apply/success | success.html | Submission confirmed page with application number |
| /admin | admin.html | Simple admin view to see submitted applications (for testing) |

---

## 3. Repository File Structure

This is a completely separate repository from the main FormAssist project. Name the repo: mu-scholarship-portal

```
mu-scholarship-portal/
|
+-- app.py                      # Flask entry point
+-- config.py                   # Secret key, DB config, upload limits
+-- requirements.txt            # Flask, SQLAlchemy, Pillow, etc.
+-- .env                        # Secret key (never commit)
+-- .gitignore
|
+-- routes/
|   +-- home_routes.py          # Landing page
|   +-- apply_routes.py         # All form steps
|   +-- admin_routes.py         # Admin view
|   +-- api_routes.py           # JSON endpoints for FormAssist to call
|
+-- services/
|   +-- session_service.py      # Save/load form progress in Flask session
|   +-- submission_service.py   # Save final submission to DB
|   +-- validation_service.py   # Server-side field validation
|
+-- database/
|   +-- db.py                   # SQLAlchemy init
|   +-- application_model.py    # Scholarship application table
|
+-- templates/
|   +-- base.html               # Master layout with navbar, progress bar
|   +-- index.html              # Landing page
|   +-- apply_step1.html        # Personal details
|   +-- apply_step2.html        # Academic details
|   +-- apply_step3.html        # Bank details
|   +-- apply_step4.html        # Document uploads
|   +-- apply_step5.html        # Review page
|   +-- success.html            # Confirmation page
|   +-- admin.html              # Admin dashboard
|
+-- static/
|   +-- css/
|   |   +-- style.css           # Global styles
|   |   +-- form.css            # Form step styles
|   +-- js/
|   |   +-- steps.js            # Progress bar logic
|   |   +-- upload_preview.js   # File upload previews
|   |   +-- validation.js       # Client-side validation
|   +-- images/
|       +-- mu_logo.png         # Mumbai University logo placeholder
|       +-- gov_banner.png      # Top banner image
|
+-- uploads/                    # Uploaded documents stored here
|   +-- .gitkeep
|
+-- tests/
    +-- test_routes.py
```

```
+-- test_validation.py
```

---

# 4. Landing Page — index.html

This is the first page a user sees. It must look like a real scholarship portal with the following sections:

## 4.1 Header / Hero Section

- Mumbai University crest/logo on the left
- Title: 'Dr. Babasaheb Ambedkar Merit-cum-Means Scholarship 2024-25'
- Tagline: 'Empowering Students of Maharashtra through Financial Assistance'
- Two buttons: 'Apply Now' (links to /apply) and 'Check Status' (dummy link)

## 4.2 Scholarship Details Section

- Award amount: Up to Rs. 50,000 per year
- Duration: 1 year, renewable based on academic performance
- Last date to apply: 31st March 2025 (hardcoded — fake)
- Display as an info card with icons

## 4.3 Eligibility Criteria Section

- Must be enrolled in Mumbai University affiliated college
- Annual family income must not exceed Rs. 8,00,000
- Minimum 60% marks in previous year
- Must be a domicile of Maharashtra
- Display as a checklist-style card

## 4.4 Required Documents Section

Display a grid of document cards showing what the applicant must upload. This is important — it tells the FormAssist team what documents are expected for OCR testing:

| Document | Format accepted | Max size | Purpose in FormAssist test |
|---|---|---|---|
| Aadhaar Card | JPG, JPEG, PDF | 200 KB | OCR extracts 12-digit Aadhaar number |
| PAN Card | JPG, JPEG, PDF | 200 KB | OCR extracts 10-character PAN number |
| College ID Card | JPG, JPEG | 150 KB | OCR extracts roll number |
| Income Certificate | PDF | 500 KB | OCR extracts income amount |
| Marksheet (Previous Year) | PDF, JPG | 500 KB | OCR extracts percentage/CGPA |
| Bank Passbook Front Page | JPG, JPEG | 200 KB | OCR extracts account number and IFSC |
| Passport Photo | JPG | 50 KB | Uploaded as-is, no OCR needed |
| Domicile Certificate | PDF | 300 KB | OCR extracts district name |

# 5. Multi-Step Form — All Fields

## 5.1 Step 1 — Personal Details (/apply)

Route: GET renders form, POST saves to session and redirects to /apply/step2

Fields to include:

| Field label | Input type | name attribute | Validation rule |
|---|---|---|---|
| Full Name (as per Aadhaar) | text | full_name | Required, letters and spaces only |
| Date of Birth | date | dob | Required, must be 17-30 years old |
| Gender | select | gender | Required: Male / Female / Other |
| Category | select | category | Required: General / OBC / SC / ST / NT |
| Mobile Number | tel | mobile | Required, exactly 10 digits |
| Email Address | email | email | Required, valid email format |
| Aadhaar Number | text | aadhaar_number | Required, exactly 12 digits — THIS IS THE OCR TARGET FIELD |
| PAN Number | text | pan_number | Required, format: AAAAA0000A — THIS IS THE OCR TARGET FIELD |
| Permanent Address | textarea | address | Required |
| City | text | city | Required |
| District | select | district | Required, dropdown of Maharashtra districts |
| PIN Code | text | pincode | Required, exactly 6 digits |

> ⚠ *The aadhaar_number and pan_number fields are the most important ones — FormAssist will use OCR on uploaded documents to fill these in automatically. Give these fields distinct IDs: id='aadhaar_number' and id='pan_number' so FormAssist's autofill.js can target them.*

## 5.2 Step 2 — Academic Details (/apply/step2)

Route: GET renders form with session data, POST saves to session and redirects to /apply/step3

| Field label | Input type | name attribute | Validation rule |
|---|---|---|---|
| College Name | text | college_name | Required |
| University Enrollment Number | text | enrollment_no | Required, alphanumeric |
| Course Name | text | course_name | Required e.g. B.Sc Computer Science |
| Current Year of Study | select | study_year | Required: First / Second / Third / Fourth |

| | | | |
|---|---|---|---|
| Admission Type | select | admission_type | Required: Regular / Part-time / Distance |
| Percentage in Last Exam | number | last_percentage | Required, 0-100, minimum 60 |
| Year of Passing Last Exam | select | passing_year | Required, dropdown 2018-2024 |
| Board / University of Last Exam | text | last_university | Required |
| Gap Year (if any) | select | gap_year | Yes / No |
| Reason for Gap (if yes) | textarea | gap_reason | Required only if gap_year is Yes |

## 5.3 Step 3 — Bank & Financial Details (/apply/step3)

Route: GET renders form with session data, POST saves to session and redirects to /apply/step4

| Field label | Input type | name attribute | Validation rule |
|---|---|---|---|
| Annual Family Income (Rs.) | number | annual_income | Required, max 800000 |
| Number of Earning Members | number | earning_members | Required, 1-10 |
| Bank Account Number | text | bank_account | Required, 9-18 digits |
| Confirm Account Number | text | bank_account_confirm | Must match bank_account |
| Bank Name | text | bank_name | Required |
| Branch Name | text | branch_name | Required |
| IFSC Code | text | ifsc_code | Required, format: AAAA0000000 |
| Account Holder Name | text | account_holder | Required, must match full_name |
| Account Type | select | account_type | Savings / Current |
| Father's Name | text | father_name | Required |
| Father's Occupation | text | father_occupation | Required |
| Mother's Name | text | mother_name | Required |

## 5.4 Step 4 — Document Upload (/apply/step4)

Route: GET renders upload form, POST handles each file, saves to uploads/ folder, redirects to /apply/step5

This is the most critical step for FormAssist testing. Each upload input must:

- Show the file name and size after selection (JavaScript)
- Show a preview thumbnail for image files
- Display the allowed formats and max size clearly next to each field
- NOT auto-submit — let the user choose all files then click Next

Upload fields:

| Field label | name attribute | accept attribute | Max size | OCR use |
|---|---|---|---|---|
| Aadhaar Card * | aadhaar_doc | .jpg,.jpeg,.pdf | 200KB | Extract Aadhaar number |
| PAN Card * | pan_doc | .jpg,.jpeg,.pdf | 200KB | Extract PAN number |
| College ID Card * | college_id_doc | .jpg,.jpeg | 150KB | Extract roll number |
| Income Certificate * | income_doc | .pdf | 500KB | Extract income figure |
| Previous Year Marksheet * | marksheet_doc | .pdf,.jpg,.jpeg | 500KB | Extract percentage |
| Bank Passbook (Front Page) * | passbook_doc | .jpg,.jpeg | 200KB | Extract account/IFSC |
| Passport Photo * | photo_doc | .jpg | 50KB | No OCR — display only |
| Domicile Certificate * | domicile_doc | .pdf | 300KB | Extract district |

⚠ *Backend must save uploaded files to uploads/<session_id>/<filename> so different test users don't overwrite each other's files.*

### 5.5 Step 5 — Review Page (/apply/step5)

This page shows all the data collected across all steps for the user to confirm before final submission. It must:

- Display all personal, academic, bank details in read-only cards
- Show thumbnails / filenames for each uploaded document
- Have an Edit button next to each section that links back to that step
- Have a final Declaration checkbox: 'I hereby declare that all information provided is true and correct'
- Have a Submit Application button that POSTs to /apply/submit

⚠ *The review page is important for FormAssist demo — it shows the evaluator that autofilled data (Aadhaar no., PAN no.) appeared correctly in the form.*

---

## 6. Backend — Flask Routes & Logic

### 6.1 app.py — Entry Point

```
from flask import Flask
from database.db import db
from routes.home_routes import home_bp
from routes.apply_routes import apply_bp
from routes.admin_routes import admin_bp
from routes.api_routes import api_bp

app = Flask(__name__)
app.config.from_object('config.Config')
```

```
db.init_app(app)
app.register_blueprint(home_bp)
app.register_blueprint(apply_bp, url_prefix='/apply')
app.register_blueprint(admin_bp, url_prefix='/admin')
app.register_blueprint(api_bp,   url_prefix='/api')

if __name__ == '__main__':
    with app.app_context(): db.create_all()
    app.run(debug=True, port=5001)
```

⚠ *Run on port 5001 so it doesn't clash with the main FormAssist app on port 5000.*

## 6.2 Session-Based Form Progress

Since this is a multi-step form, use Flask's session to store data between steps. This way the user can go back and forward without losing data.

```
# services/session_service.py
from flask import session

def save_step(step_name: str, data: dict):
    if 'form_data' not in session:
        session['form_data'] = {}
    session['form_data'][step_name] = data
    session.modified = True

def get_all_data() -> dict:
    return session.get('form_data', {})

def clear_form():
    session.pop('form_data', None)
```

## 6.3 apply_routes.py — All Form Step Routes

Each step follows the same GET/POST pattern:

```
@apply_bp.route('/', methods=['GET','POST'])
def step1():
    if request.method == 'POST':
        data = request.form.to_dict()
        errors = validate_step1(data)
        if errors:
            return render_template('apply_step1.html', errors=errors, data=data)
        save_step('step1', data)
        return redirect(url_for('apply.step2'))
    saved = get_all_data().get('step1', {})
    return render_template('apply_step1.html', data=saved)
```

Repeat this exact pattern for step2, step3. Step 4 (uploads) is different — see below.

## 6.4 File Upload Handling — Step 4

```
@apply_bp.route('/step4', methods=['GET','POST'])
```

```
def step4():
    if request.method == 'POST':
        uploaded = {}
        for field_name in ['aadhaar_doc','pan_doc','college_id_doc',
                           'income_doc','marksheet_doc','passbook_doc',
                           'photo_doc','domicile_doc']:
            file = request.files.get(field_name)
            if file and file.filename:
                filename = secure_filename(file.filename)
                save_path = os.path.join(app.config['UPLOAD_FOLDER'], filename)
                file.save(save_path)
                uploaded[field_name] = filename
        save_step('step4', uploaded)
        return redirect(url_for('apply.step5'))
    return render_template('apply_step4.html')
```

## 6.5 Final Submission Route

```
@apply_bp.route('/submit', methods=['POST'])
def submit():
    all_data = get_all_data()
    app_no = generate_application_number()  # e.g. MU-SCH-2024-00142
    # Save to database
    new_app = Application(
        application_no = app_no,
        full_name      = all_data['step1']['full_name'],
        aadhaar_number = all_data['step1']['aadhaar_number'],
        pan_number     = all_data['step1']['pan_number'],
        data_json      = json.dumps(all_data)
    )
    db.session.add(new_app)
    db.session.commit()
    clear_form()
    return render_template('success.html', app_no=app_no)
```

## 6.6 API Routes — For FormAssist Integration

These JSON endpoints let the FormAssist team demonstrate their AI filling in this form programmatically. This is the bridge between the two projects.

| Endpoint | Method | Purpose | Returns |
|---|---|---|---|
| /api/form-fields | GET | Returns list of all form fields with their IDs and types | JSON: [{id, name, type, step}] |
| /api/submit-autofill | POST | Accepts pre-filled form data from FormAssist and saves it | JSON: {success, application_no} |
| /api/applications | GET | List all submitted applications (admin use) | JSON: [{app_no, name, aadhaar, pan, submitted_at}] |

⚠ *The /api/form-fields endpoint is especially useful — FormAssist can call it to know exactly what field IDs exist on this site, then populate them.*

### 6.7 Database Model

```
# database/application_model.py
class Application(db.Model):
    id             = db.Column(db.Integer, primary_key=True)
    application_no = db.Column(db.String(30), unique=True)
    full_name      = db.Column(db.String(150))
    aadhaar_number = db.Column(db.String(12))
    pan_number     = db.Column(db.String(10))
    email          = db.Column(db.String(120))
    mobile         = db.Column(db.String(10))
    data_json      = db.Column(db.Text)  # full form data as JSON
    submitted_at   = db.Column(db.DateTime, default=datetime.utcnow)
    status         = db.Column(db.String(20), default='Submitted')
```

---

## 7. Validation Rules

Validation happens in two places: client-side in validation.js (instant feedback) and server-side in validation_service.py (security). Both must be implemented.

| Field | Client-side check | Server-side check |
|---|---|---|
| full_name | Not empty, only letters and spaces | Strip whitespace, max 150 chars |
| aadhaar_number | Exactly 12 digits, no spaces | Must be 12 digits, store as string not int |
| pan_number | Regex: [A-Z]{5}[0-9]{4}[A-Z]{1} | Upper-case before saving, regex check |
| mobile | Exactly 10 digits, starts with 6-9 | Regex: ^[6-9]\d{9}$ |
| email | HTML5 email type | Python: re.match email pattern |
| last_percentage | Number 0-100, minimum 60 | Float check, reject below 60 |
| annual_income | Number, max 800000 | Integer check, reject above 800000 |
| bank_account | 9-18 digits only | Length 9-18, digits only |
| ifsc_code | Regex: [A-Z]{4}0[A-Z0-9]{6} | Upper-case, regex check |
| pincode | Exactly 6 digits | Digits only, length 6 |
| File uploads | Check file extension and size in JS | Check MIME type server side, reject others |

---

## 8. Step Progress Bar (base.html)

Include a visible 5-step progress bar at the top of every apply page. Users can click on completed steps to go back.

```
<!-- In base.html, inside the <header> section -->
<div class='progress-bar'>
  <div class='step {% if current_step >= 1 %}active{% endif %}'>
```

```
    <span>1</span><label>Personal</label>
  </div>
  <div class='step {% if current_step >= 2 %}active{% endif %}'>
    <span>2</span><label>Academic</label>
  </div>
  <div class='step {% if current_step >= 3 %}active{% endif %}'>
    <span>3</span><label>Financial</label>
  </div>
  <div class='step {% if current_step >= 4 %}active{% endif %}'>
    <span>4</span><label>Documents</label>
  </div>
  <div class='step {% if current_step >= 5 %}active{% endif %}'>
    <span>5</span><label>Review</label>
  </div>
</div>
```

Each route passes current_step variable to the template. Step 1 passes current_step=1, Step 2 passes current_step=2, and so on.

---

## 9. Admin Page (/admin)

A simple password-protected page to view all submitted applications. This is used during FormAssist testing/demo to verify submissions came through correctly.

- Basic HTTP password: admin / formassist123 (hardcoded for testing)
- Show a table of all submissions: Application No., Name, Aadhaar, PAN, Submitted At, Status
- Click on any row to see full submitted data in a modal
- Aadhaar and PAN numbers should be partially masked in the table: XXXX-XXXX-1234
- Show unmasked version only on row click inside modal

> ⚠ *This page is only for the FormAssist team to verify OCR extracted the correct numbers. Do not expose it publicly during any live demo.*

---

## 10. How FormAssist Will Use This Site

This explains the connection between the main FormAssist project and this test site. Share this section with all team members.

### The Full Demo Flow

1. User opens FormAssist (main project, runs on port 5000)
2. User uploads their Aadhaar card image to FormAssist
3. FormAssist OCR engine reads the image and extracts the 12-digit Aadhaar number
4. FormAssist NLP processor also extracts name, DOB, address from the document
5. User clicks 'Fill Scholarship Form' in FormAssist
6. FormAssist calls GET /api/form-fields on this site (port 5001) to get field list
7. FormAssist's autofill maps extracted data to field IDs on this site
8. User is redirected to this scholarship site with form fields pre-filled
9. User reviews the pre-filled data including the auto-filled Aadhaar and PAN numbers

10. User submits — evaluator checks /admin to confirm correct data was submitted

### Field ID Mapping — Critical

The FormAssist team's autofill model returns field names. This site's form must use matching IDs. Do not change these IDs without telling the FormAssist backend person:

| FormAssist predict_fields() key | This site's field id= | Step |
|---|---|---|
| full_name | full_name | Step 1 |
| dob | dob | Step 1 |
| mobile_number | mobile | Step 1 |
| email_address | email | Step 1 |
| aadhaar_number | aadhaar_number | Step 1 — OCR FILLED |
| pan_number | pan_number | Step 1 — OCR FILLED |
| permanent_address | address | Step 1 |
| college_name | college_name | Step 2 |
| bank_account | bank_account | Step 3 |
| ifsc_code | ifsc_code | Step 3 |

---

## 11. Setup & Running Instructions

### To run this site locally:

11. Clone the repo: git clone https://github.com/your-org/mu-scholarship-portal
12. Create virtual environment: python -m venv venv
13. Activate: venv\Scripts\activate (Windows) or source venv/bin/activate (Mac/Linux)
14. Install packages: pip install -r requirements.txt
15. Create .env file with SECRET_KEY=any-random-string
16. Run: python app.py
17. Open browser: http://localhost:5001

### requirements.txt contents:

```
Flask==3.0.0
Flask-SQLAlchemy==3.1.1
Werkzeug==3.0.1
python-dotenv==1.0.0
Pillow==10.2.0
```

---

## 12. Git Workflow

- Repository name: mu-scholarship-portal (separate from main FormAssist repo)
- Since this is a single-person project, working directly on main is acceptable
- Commit after completing each step: 'Add Step 1 personal details form'
- Never commit the uploads/ folder contents or .env file
- Add a README.md explaining how to run it and what it is for

**Good commit message examples:**

```
git commit -m 'Add landing page with eligibility criteria'
git commit -m 'Add Step 1 personal details form with validation'
git commit -m 'Add file upload step with preview and size check'
git commit -m 'Add /api/form-fields JSON endpoint for FormAssist'
git commit -m 'Add admin dashboard with masked Aadhaar display'
```

---

## 13. Final Checklist Before Handing Over to FormAssist Team

Before the main FormAssist team starts integration testing, verify all of these:

- Landing page loads at http://localhost:5001
- All 5 form steps work and data persists in session when going back
- File uploads save to uploads/ folder correctly
- Review page shows data from all previous steps
- Success page shows a generated application number
- Admin page at /admin shows submitted applications
- GET /api/form-fields returns correct JSON with all field IDs
- POST /api/submit-autofill accepts and saves form data
- Field IDs match the mapping table in Section 10 exactly
- Aadhaar field id='aadhaar_number' and PAN field id='pan_number' confirmed
- Site runs on port 5001 (not 5000)
- README.md is written and clear